OASIS: Regime-Aware Feedback Calibration for Optimizer Marginal Statistics

Author Names Redacted for Review[a]

[a] *Affiliation Redacted for Review*

Abstract

Cost-based optimizers rely on marginal statistics that become stale between refreshes. Query feedback exposes local selectivity errors, but a learned or heuristic repair is not automatically safe to expose to a planner: composition, join estimation, and plan selection can amplify a locally inconsistent marginal. Our thesis is that optimizer-facing statistics need a regime-aware feedback-consistency *calibration layer*—and that this layer, not the prior generator, is where the deployment problem lives.

OASIS separates marginal prior generation (Stage 1) from optimizer-facing calibration (Stage 2); the contribution is Stage 2. Hard feedback projection, adapted from ISOMER/IPF, enforces exact feedback consistency and is safest for joins and the planner, whereas a new consistency-gated soft projection generalizes better on future single-column predicates. Because no operator dominates, a residual-gated router selects between them per case from a deployment-visible signal; a banded relaxation is a negative control isolating the gain to regime-aware routing rather than looser constraints.

OASIS improves every member of a six-estimator composition family by 22%–38% and keeps raw learned marginals from regressing in a FactorJoin join kernel. In a 12-configuration PostgreSQL planner-only study, it lowers aggregate row-estimation Q-error from 27.748 to 2.360 and new plan deviations from 13.8% to 1.1%. A three-seed TPC-H SF10 refresh-drift study shows calibrated statistics reproduce fresh-statistics behavior—scan Q-error 2.0 versus stale 5.2, and warm-cache runtime within a few percent of a full `ANALYZE`—without the table scan. The same calibration improves stale, heuristic, STHoles, and QuickSel-H priors, so it is reusable; the learned prior is useful but secondary.

Keywords: Statistics correction, Cardinality estimation, Histogram

calibration, Cost-based optimizer, Query feedback, Online learning

1. Introduction

Cost-based optimizers (CBOs) choose query plans from estimates of predicate selectivity, intermediate cardinality, and operator cost [1, 2]. In many analytical database systems, these estimates rely on column-level statistics such as histograms, most-common-value lists, and density summaries [3, 4, 5, 6]. The statistics are periodically refreshed through `ANALYZE`-style procedures that read a full or sampled view of the table [7]. This maintenance strategy is simple and robust, but it leaves a persistent staleness gap: tables continue to receive inserts, deletes, and updates while the optimizer keeps planning with the last statistics snapshot. As the gap widens, stale histograms induce systematic selectivity errors that can propagate through the optimizer's cost model and lead to poor plan choices [8, 9, 10, 11].

Existing remedies operate at different layers of the optimization stack. Frequent `ANALYZE` reduces staleness but increases scan overhead. Learned cardinality estimators predict point cardinalities for particular queries and often require raw data, query workloads, or table-specific training [12, 13, 14, 10]. Adaptive and plan-level feedback systems can re-optimize, adjust plan choices, or learn hints after observing execution behavior [15, 16, 17, 18, 19], but they do not repair the underlying statistics objects consumed by the optimizer. This paper focuses on a complementary layer: *statistics-level correction*. Rather than replacing the optimizer or learning complete query cardinalities, we update the column statistics supplied to the existing CBO. We state the evaluation boundary up front: the study is primarily *planner-facing*. We measure the corrected statistics object and the row estimates and plan shapes a real PostgreSQL optimizer derives from it, and add one deliberately narrow TPC-H runtime check showing that calibrated statistics reproduce fresh-statistics execution time. We do not claim broad query-runtime superiority over the stale baseline.

The key observation is that a DBMS already obtains useful feedback while executing ordinary queries. For a filter predicate, the optimizer's estimated selectivity and the executor's observed selectivity expose a local discrepancy between the stale histogram and the current data distribution. The tempting response is to repair the histogram directly from those observations. However, the deployment problem is sharper than producing a marginal that

looks better on average: a repaired marginal must also be reliable near the predicates that were just observed, because downstream estimators and planners may amplify local one-dimensional mistakes into larger joint-cardinality or plan-shape errors.

This paper treats stale marginal-statistics correction as two coupled tasks: *prior generation* and *feedback calibration*. OASIS instantiates the first task with a compact pooled MLP that supplies a post-drift marginal prior. The prior is useful—under a fixed projection operator it lowers held-out future-predicate Q-error from 1.536 (projection from stale, i.e., ISOMER) to 1.446—but prior quality alone is not sufficient. A raw repaired marginal can look globally plausible while violating the current feedback window, and our downstream experiments show that this local mismatch makes it weak for composition and actively harmful in a bilinear join kernel.

The main contribution of OASIS is therefore the second task: an optimizer-facing Stage-2 calibration layer. The layer starts from a marginal prior, whether learned or heuristic, and returns a feedback-calibrated marginal statistics object without changing the optimizer's search procedure or cost model. Its hard operator is an ISOMER/IPF-style projection that exactly matches the active feedback constraints and is the safe choice for join and planner-facing use. Exact matching, however, is not uniformly best: on future single-column predicates it can overfit the feedback window. We therefore add a consistency-gated soft projection that regularizes toward the prior while down-weighting feedback contradicted by the most recent observations. Because no single operator dominates, the deployed Stage 2 is a residual-gated *Router* that chooses among stale, local-feedback, OASIS-noProj, OASIS, and soft-projected marginals using only the current feedback-window residual. A banded relaxation of the equality constraints is a negative result: widening the bands monotonically worsens accuracy, isolating the gain to regime-aware calibration and routing rather than to looser matching.

To state the novelty boundary precisely: the hard projection is adapted from ISOMER/IPF feedback consistency and is *not* claimed as new, and the learned prior is a useful but *secondary* component. The new contributions are (i) the separation of prior generation from optimizer-facing calibration; (ii) the finding that the right calibration is *regime-dependent*—exact projection for joins and the planner, softer projection for future single-column predicates; (iii) the consistency-gated soft projection that down-weights contradicted feedback; and (iv) the residual-gated Router that operationalizes

this regime-dependence from a deployment-visible signal. A Stage-1 swap confirms the layer is not a patch for one model: feeding stale statistics, simple heuristics, STHoles, QuickSel-H, and the OASIS MLP through the same Stage-2 protocol, Router improves every raw prior in aggregate and leaves final Q-error clustered near 1.39–1.44. The learned Stage 1 helps mainly where feedback is too sparse to pin the distribution, while the reusable contribution is the calibration layer that makes imperfect marginals safe to consume.

The evaluation follows this scope. We evaluate the mechanism where it acts: future selectivity estimates, structural histogram error, downstream marginal composition—including a six-estimator composition family and the FactorJoin join kernel that consume OASIS as a drop-in marginal source—and real PostgreSQL planner estimates and plan shapes. The PostgreSQL study is planner-only: true rows are obtained with `COUNT(*)`, and optimizer-facing evidence comes from `EXPLAIN (FORMAT JSON)` under injected statistics states, without measuring query runtime. To reduce dependence on a single drift simulator, we also evaluate out-of-distribution drift families and benchmark-inspired DML traces with explicit insert, update, and delete events, plus a public production HTTP telemetry trace used as an append-only event-table case study.

The contributions are:

- We formulate feedback-driven marginal-statistics repair as a DBMS-facing calibration problem: given a stale histogram, a causal feedback window, and a marginal prior from any repair source, produce optimizer-consumable statistics without rescanning the table or changing the optimizer. We instantiate a PostgreSQL-style adapter that maps MCV+histogram statistics to a canonical quantile/CDF view and back.
- We design OASIS around a reusable Stage-2 calibration layer and show its gains are *regime-dependent*. The layer reuses a hard feedback-consistency projection adapted from ISOMER/IPF (not claimed as new) and adds two new components: a consistency-gated soft projection that down-weights contradicted feedback, and a residual-gated Router that selects per case from deployment-visible feedback residuals. A banded projection is reported as a negative result, separating routing/calibration gains from simple constraint relaxation.
- We keep Stage 1 deliberately lightweight and bounded in the story: under a fixed projection operator its learned prior improves future-predicate Q-error over projection from stale statistics, while a Stage-1 swap shows that Stage 2 is estimator-agnostic across stale, heuristic, STHoles, QuickSel-H,

and OASIS-MLP priors.

- We show that calibration is necessary for downstream and planner safety. Raw repaired marginals are not feedback-consistent, give weak and unstable composition gains, and are actively worse than stale in the FactorJoin join kernel; calibrated marginals improve every estimator in a six-member composition family and reduce PostgreSQL new plan deviations from 13.8% to 1.1%.
- We give a primarily optimizer-facing evaluation spanning single-column drift, structural and initial-distribution generalization, a six-estimator composition family, the FactorJoin join kernel, a 12-configuration PostgreSQL planner-only statistics-injection study, deployment-safety checks under sparse, noisy, synthetic trace-grounded, and public production-telemetry feedback, plus a narrow TPC-H no-regression runtime sanity check.

2. Background and Problem Formulation

2.1. Column Statistics and the Staleness Gap

We model a column histogram as an equi-depth quantile summary with B buckets. The internal boundaries $v_1, \ldots, v_{B-1}$ correspond to fixed quantile levels $p_1, \ldots, p_{B-1}$. A range predicate such as $x < c$ is estimated by evaluating the piecewise-linear CDF implied by those boundaries. Although real DBMS statistics may combine histograms with auxiliary summaries such as most-common-value lists, density vectors, or null fractions, OASIS operates on a canonical one-dimensional CDF view.

Let H_0 denote the stale histogram captured at the last statistics refresh and H^* denote the current post-drift distribution. For a predicate σ, the optimizer derives an estimate $\hat{s}_{H_0}(\sigma)$ while the executor can later reveal an actual selectivity $s^*(\sigma)$. We measure selectivity error using Q-error:

$$\mathrm{QErr}(\hat{s}, s^*) = \max\left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}}\right). \tag{1}$$

2.2. Statistics Correction

Given a stale histogram H_0 and an observation window $\mathcal{O} = \{o_1, \ldots, o_K\}$, where each observation records a predicate type, predicate boundary, estimated selectivity, and actual selectivity, the goal is to produce corrected statistics $\hat{H}$ such that future predicates are estimated more accurately:

$$\mathbb{E}_\sigma\left[\mathrm{QErr}\left(\mathrm{CDF}_{\hat{H}}(\sigma), s^*_\sigma\right)\right] < \mathbb{E}_\sigma\left[\mathrm{QErr}\left(\mathrm{CDF}_{H_0}(\sigma), s^*_\sigma\right)\right].$$

The expectation is over future predicates from the workload distribution.

The solution must satisfy three deployment constraints. First, it must avoid table rescans, deriving $\hat{H}$ only from H_0 and recent feedback. Second, it must be cheap enough to run near query-planning time. Third, it must degrade gracefully when feedback is sparse or uninformative, preserving the stale prior rather than applying large unsupported corrections.

2.3. Scope

OASIS targets single-column histogram staleness. It does not repair multi-column correlation errors, join-key dependence, physical design problems, or cost-model calibration. These terms remain in the optimizer's error budget even if the single-column histogram is accurate. The role of OASIS is to improve the marginal statistics that many downstream estimators and optimizers already consume.

3. OASIS Design

3.1. System Overview

OASIS sits between the statistics store and the optimizer. When the optimizer requests a histogram, OASIS reconstructs a canonical quantile representation, retrieves a recent observation window for the column, obtains a marginal prior from a Stage-1 repair source, calibrates that prior with the Stage-2 router, and returns an engine-native statistics object to the CBO. Query execution remains unchanged except for lightweight per-conjunct feedback collection.

Naming.. To keep the variants legible, we fix one term per concept and collect them here. At the system level, *OASIS* names our overall approach: a learned Stage-1 prior plus the Stage-2 calibration layer. As a concrete method label—and as the "OASIS" column in every result table—**OASIS** denotes the default deployed instantiation: the learned prior calibrated by **hard projection**. **OASIS-noProj** is its Stage-1-only ablation (the raw prior, no calibration). The Stage-2 layer provides two calibration *operators*—hard projection and **Soft**, the consistency-gated soft projection—and two selection *policies* that choose per case among candidate marginals: **Router** over {stale, ISOMER, OASIS-noProj, OASIS, Soft}, and **Hybrid**, the same router without the Soft candidate. Router is the deployed Stage 2; it is a policy over the

candidates above, not a separate marginal, so it is never itself called "OASIS." The external baselines are the **stale** histogram and **ISOMER** (hard projection initialized from the stale histogram rather than the learned prior). **Banded projection** is a negative control, not a deployed method.

3.2. Feedback Collection and Integration

OASIS requires two integration points. A query-completion listener records predicate-level feedback from executor row-count instrumentation. A statistics-assembly hook intercepts the histogram before it reaches the optimizer and substitutes the corrected version. Because the correction is inserted at the statistics layer, the optimizer's search procedure and cost model remain unchanged.

3.3. Statistics-Format Conversion

Many DBMSs do not expose a standalone editable equi-depth histogram. PostgreSQL, for example, couples histogram boundaries with MCV lists and other per-column summaries. We instantiate the interface for PostgreSQL-style statistics and design it to isolate engine-specific adapters. The adapter has three phases: reconstruct a canonical distribution view from the native object, apply correction on the canonical view, and decompose the corrected view back into native statistics fields. This paper validates the PostgreSQL-style MCV+histogram instantiation; other engines require their own adapter logic.

3.4. Stage 1: Marginal Prior Generation

The Stage-1 component supplies a marginal prior to the calibration layer. It is deliberately lightweight and is only *one* of several interchangeable prior sources: Stage 2 is not tied to it, and Section 5.4 feeds stale, heuristic, STHoles, and QuickSel-H marginals through the same calibration protocol. We describe the learned instantiation here because it is the prior used in most experiments, but the paper's claims do not depend on it.

3.4.1. Feature Representation

The prior generator consumes a fixed tensor with four blocks: normalized stale quantile boundaries, lightweight metadata, up to K recent predicate-feedback observations, and a mask for valid observation slots. In our implementation $B = 10$ and $K = 16$, which keeps the representation small enough for planning-time use.

3.4.2. Prior Generator

The main OASIS instantiation uses a compact pooled MLP with a residual head that predicts corrections to internal quantile boundaries; the output is clamped and monotonized to form a valid CDF. The model is trained offline on simulation-derived compound drift and then reused across columns without per-table retuning. Because Stage 2 accepts any prior, this learned generator can be replaced by a cheaper heuristic wherever its extrapolation benefit is not needed.

3.5. Stage 2: Regime-Aware Feedback Calibration

The Stage-2 component takes a marginal prior and calibrates it against the current feedback window before the marginal reaches a downstream estimator or a DBMS planner. The division of labor is the central idea: prior generation controls how far the correction can extrapolate beyond observed predicates, while Stage 2 controls whether the statistics object is safe to compose, join, or inject into a planner. Stage 2 remains a single-column calibration layer; it does not estimate multi-column dependence. The rest of this section presents the two calibration operators—a hard projection and a consistency-gated soft projection—a banded negative control that rules out a tempting alternative, and the residual-gated router that selects among them and constitutes the deployed Stage 2.

3.5.1. Hard Projection

The hard operator is not new: it adapts the ISOMER/IPF-style feedback-consistency projection [20] to act on a supplied marginal prior. Our contribution is not the projection itself but its use as a calibration stage over an arbitrary prior, and the regime analysis that decides when it is the right operator. Concretely, it builds the interval partition induced by the active feedback predicates, initializes cell masses from the supplied marginal prior, and performs cyclic I-projections so that every active predicate's interval mass matches its observed selectivity exactly. When sequential drift makes the active set inconsistent, the oldest constraints are dropped first, an implicit stale-feedback filter. We denote this default two-stage output—the learned Stage-1 prior calibrated by hard projection—**OASIS**; it is the safe form for downstream composition and the DBMS planner.

3.5.2. Consistency-Gated Soft Projection

Exact matching is not always optimal. On *future* single-column predicates, matching a handful of recent feedback intervals exactly can overfit them; a softer calibration that regularizes toward the learned marginal generalizes better. We therefore add a soft Stage-2 operator that minimizes $\mathrm{KL}(p \,\|\, p_{\text{prior}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ over the same partition, where A_j is a feedback interval mask and y_j its observed selectivity. The crucial component is the weight w_j: rather than discarding old feedback by age, we discard it by *inconsistency with the current data state*. We fit a reference distribution to the most recent observations, score each constraint by its residual against that reference, and set $w_j = \exp(-(\text{conflict}_j/\tau)^2)$, so feedback that the recent observations contradict is suppressed while old-but-still-consistent feedback is retained. This keeps the single-column generalization benefit of soft calibration without softly fitting stale, contradicted constraints.

3.5.3. A Negative Result: Banded Projection

A natural alternative is to relax the equality constraints into confidence bands $y_j \pm \delta_j$ and I-project the learned prior onto that polytope, so that $\delta_j = 0$ recovers hard projection. This does not help: widening the band monotonically worsens single-column accuracy (Section 5.3). The soft operator's advantage comes from a better-conditioned cell-level optimization—it attains *lower* feedback residual than hard projection on single-column data—not from relaxed matching, so relaxation is the wrong axis.

3.5.4. The Router

No single operator dominates: hard projection is best on the join kernel and the planner, soft projection is best on future single-column and optimizer-facing predicates. We resolve this with a residual-gated *Router* that selects, per case, the candidate with the lowest feedback residual on the current observation window—a deployment-visible signal with no fresh labels—among the stale prior, ISOMER, OASIS-noProj, OASIS (hard projection of the learned prior), and Soft (consistency-gated soft projection). Because the gate is a residual minimizer over a pool that contains hard projection, the router can only improve on the residual signal, and it reduces to hard projection/ISOMER on workloads (joins, the PostgreSQL planner) where soft projection has higher residual. Router, not any single operator, is the deployed Stage 2; its no-soft variant Hybrid (over stale, ISOMER, OASIS-noProj, and OASIS) remains available where the Soft candidate is unnecessary.

4. Experimental Methodology

The evaluation is organized as a ladder rather than as independent stress tests. We first establish the marginal object OASIS produces: the statistics-format adapter is lossless enough for planner use, the learned prior contains useful drift signal, and Stage 2 determines which calibrated marginal should be exposed. We then test the two possible objections raised by a Stage-2-centric story: the calibration layer should work with priors beyond the OASIS MLP, and the learned prior should still help when the projection operator is held fixed. After that, we widen the environment from synthetic drift to deployment-inspired drift, explicit DML event streams, and a public production telemetry trace. The final experiments ask whether the calibrated marginal remains safe when consumed by downstream composition estimators, a join-cardinality kernel, PostgreSQL's native planner, and sparse or noisy feedback. This order follows the deployment path of a marginal statistic: first whether the corrected object is internally calibrated, then whether downstream estimators preserve the gain, and finally whether PostgreSQL behaves like it would under fresh statistics, before a last diagnostic stress-tests the feedback channel.

Unless otherwise stated, OASIS is trained once on Gaussian compound drift with $q \in \{1, 3, 5, 10, 15, 20\}$ and evaluated on held-out drift intensities $q \in \{1, 3, 5, 10, 15, 20, 25, 30\}$. All feedback-driven methods consume the same observation budget $K = 16$ and emit corrected quantiles on the same $B = 10$ grid. We compare against the stale prior, STHoles [21], ISOMER [20], QuickSel-H [22], and simple feedback heuristics. Metrics include Q-error, selectivity MAE, quantile MAE, feedback residual, PostgreSQL estimated-row Q-error, and plan-shape match to the fresh-statistics planner. For deployment-sensitivity diagnostics that require many perturbed runs, we also use a transparent optimizer-decision proxy: future predicates are generated from held-out distributions, each statistics state selects scan and join choices under a fixed cost proxy, and the selected choice is scored by its true proxy cost. This proxy is used only to analyze feedback-budget and feedback-noise behavior; it is not a runtime benchmark.

5. Single-Column Statistics Correction

This section stays at the level of the one-dimensional statistics object. Its purpose is not yet to claim planner benefit, but to decide what marginal

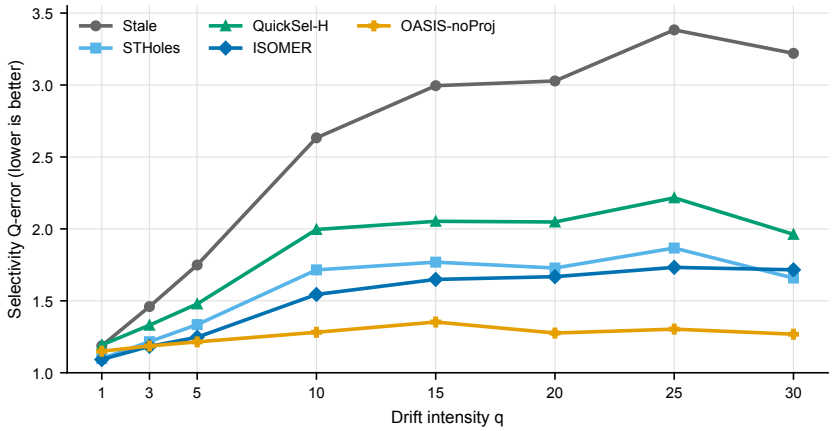

Figure 1: Single-column selectivity Q-error across drift intensity. All methods consume the same feedback budget. The figure isolates the Stage-1 prior (OASIS-noProj); the deployed calibrated form is compared in Table 2.

should be handed to downstream components. The sequence is: validate the adapter, measure the learned prior, choose the Stage-2 calibration rule, and then test whether the calibration story depends on the particular Stage-1 prior or on the training drift generator.

5.1. Format-Conversion Validation

The PostgreSQL-style MCV+histogram conversion preserves total probability mass exactly in the tested settings. Round-trip residual-quantile MAE is at machine precision, with mean 2.4×10^{-17} and maximum 9.2×10^{-17}. Selectivity estimates remain consistent before and after conversion, with global MAE 1.1×10^{-4} and worst-case error below 0.39%. With PostgreSQL-like defaults, the full round trip completes in 0.066–0.073 ms. This validates the PostgreSQL-style conversion as a low-overhead bridge between native statistics and the canonical OASIS view.

5.2. Q-Error Across Drift Intensity

Figure 1 isolates the learned prior before calibration. OASIS-noProj degrades gracefully at mild drift and becomes strongest from $q \geq 5$, reducing stale Q-error by 51.3%, 57.9%, and 60.6% at $q = 10$, 20, and 30. This establishes that the prior generator carries useful drift signal; the rest of the paper asks how Stage 2 makes that prior safe for downstream use.

Table 1: Simple heuristic baselines vs. OASIS-noProj (the learned stage-1 repair) at selected drift intensities. Results are averaged over three independent training seeds (42, 123, 456) with a different evaluation protocol than the main single-column comparison; absolute Q-Error values are therefore not directly comparable across the two protocols. Both heuristics consume the same observation window K=16. FeedAvg worsens estimation at all drift levels.

	Q-Error (↓ better)			
q	Stale	LinInterp	FeedAvg	OASIS-noProj
5	1.791	1.628	1.835	**1.238**
10	2.471	2.118	2.565	**1.255**
20	3.370	2.749	3.553	**1.296**
30	3.500	2.823	3.690	**1.345**

Table 1 shows that simple feedback heuristics are much weaker than OASIS-noProj, so the prior's gain is not merely from using feedback. Supplementary structural-MAE and initial-distribution checks show the same Stage-1 prior trend.

Table 2 adds the deployed hard-projected OASIS. Projection slightly raises aggregate single-column Q-error relative to OASIS-noProj (1.307 versus 1.254), but from $q \geq 3$ the projected form remains below ISOMER. This is the single-column cost paid for the composition, join, and planner safety established below.

5.3. Stage-2 Calibration Variants and the Router

The previous subsection used hard projection. We now study the Stage-2 calibration directly. This experiment reuses the cached drift data and the same OASIS checkpoint, and evaluates each Stage-2 variant on held-out future predicates per case (geometric-mean selectivity Q-error), together with the feedback residual on the observation window, quantile MAE against fresh statistics, and an optimizer-proxy join regret. It is a controlled comparison of calibration operators for a fixed learned prior, not a new training protocol.

Table 3 shows the regime structure. Soft (full) projection has the lowest future-predicate Q-error (1.376) and residual, but it is unsafe under sequential drift; for example, OOD batch-load Q-error rises to 1.482 versus hard projection's 1.272. Consistency-gated soft keeps much of the single-column gain (1.401) while suppressing contradicted feedback. Router, selecting per case by feedback residual over {stale, ISOMER, OASIS-noProj, OASIS, Soft},

Table 2: Single-column selectivity Q-error (↓) of OASIS versus the OASIS-noProj ablation, on in-distribution compound drift. At mild drift $q = 1$, ISOMER is lowest; from $q \geq 3$, OASIS stays below ISOMER, so the deployed hard-projected form preserves a learned single-column advantage in the moderate-to-large drift regime. The projection costs only a small amount relative to the stage-1 ablation, the price paid for the downstream composition and join safety shown in Tables 7 and 8.

q	Stale	ISOMER	OASIS-noProj	OASIS	OASIS red.%
1	1.188	**1.092**	1.150	1.111	+6%
3	1.460	1.184	1.186	**1.178**	+19%
5	1.749	1.247	1.215	**1.209**	+31%
10	2.633	1.544	**1.281**	1.393	+47%
15	2.995	1.649	**1.353**	1.409	+53%
20	3.028	1.668	**1.276**	1.364	+55%
25	3.383	1.733	**1.303**	1.412	+58%
30	3.220	1.716	**1.268**	1.379	+57%

is the best deployable variant at 1.395 and routes to soft on 44% of cases. A banded projection negative control worsens as the band widens, so the gain comes from regime-aware routing rather than relaxed matching.

The same ordering holds on the optimizer-decision proxy of Section 4: selectivity Q-error / join regret are 1.446/1.0348 for OASIS, 1.404/1.0312 for Soft, 1.430/1.0326 for the stale/ISOMER/noProj/hard Hybrid, and 1.396/1.0297 for Router—the best non-fresh result. Crucially, Router does not trade away safety: where soft projection has higher feedback residual it is never selected. On the FactorJoin kernel Router stays at hard level (join Q-error 1.0174 versus OASIS 1.024) while soft projection alone is worse (1.0367); and on the PostgreSQL plan-shape batch (Section 6.3) Router is identical to Hybrid. Router is therefore $\geq$ OASIS on single-column and optimizer-proxy accuracy and equal to the deployed Hybrid on join and planner safety.

5.4. Estimator-Agnostic Calibration

The preceding calibration study fixes the Stage-1 prior to the OASIS MLP. To test whether the Stage-2 layer is a reusable calibration mechanism rather than an OASIS-specific patch, we run a Stage-1 prior-source swap. The experiment uses the same cached drift cases, the same feedback windows, and the same future predicate protocol as Section 5.3. We vary only the marginal prior supplied to Stage 2: stale statistics, two simple feedback

Table 3: Stage-2 calibration variants on held-out future predicates (cached drift, fixed OASIS checkpoint). Lower is better. Soft (full) has the lowest selectivity Q-error and feedback residual but is unsafe under sequential drift (Section 5.6); Router is the best *deployable* variant, recovering most of the single-column gain over hard projection while remaining safe to route.

Stage-2 variant	Sel. QErr	Feedback resid.	Quantile MAE	Join regret
OASIS	1.442	0.0440	0.0519	1.0344
Soft (full)	1.376	0.0348	0.0483	1.0284
Soft (recent k=8)	1.408	0.0404	0.0471	1.0322
Soft	1.401	0.0392	0.0473	1.0313
Hybrid (no soft)	1.429	0.0395	0.0513	1.0326
Router	1.395	0.0358	0.0486	1.0302

Table 4: Stage-1 prior-source swap under a fixed Stage-2 calibration protocol. Each row supplies a different single-column marginal prior to Stage 2; lower Q-error and feedback residual are better.

Stage-1 source	Raw QE	+Hard QE	+Router QE	Router gain	Raw resid.	Router resid.
Stale prior	2.404	1.539	1.437	40.2%	0.1206	0.0383
LinInterp	2.116	1.508	1.417	33.0%	0.1072	0.0379
FeedAvg	2.474	1.544	1.437	41.9%	0.1256	0.0383
STHoles	1.572	1.436	1.394	11.3%	0.0542	0.0365
QuickSel-H	1.614	1.423	1.392	13.7%	0.0573	0.0347
OASIS MLP	1.643	1.439	1.392	15.3%	0.0643	0.0358

heuristics (LinInterp and FeedAvg), STHoles, QuickSel-H, and the OASIS MLP. For each prior we report the raw marginal, hard projection initialized from that prior, and Router over {stale, ISOMER, raw prior, hard projection, consistency-gated soft projection}.

Table 4 shows that Stage 2 is not tied to the learned OASIS prior. Hard projection improves every raw prior, and Router further improves every prior in aggregate, reducing future-predicate Q-error by 11.3%–41.9%. The routed outputs also have consistently lower feedback residuals (0.0347–0.0383). There are three small per-q QuickSel-H cases where hard projection is slightly lower than the router, so the dominance claim is aggregate/source-level rather than pointwise. The broader conclusion is direct: Stage 2 is an estimator-agnostic calibration layer, while the OASIS MLP is one useful prior generator.

Table 5: Projection-initialization ablation on held-out future predicates (in-distribution compound drift; geometric-mean selectivity Q-error). The projection operator and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **ISOMER** initializes hard projection from the stale histogram; **OASIS** initializes hard projection from OASIS-noProj; **Fresh-init** initializes from the post-drift marginal as a reference upper bound. Win frac is the fraction of held-out predicates on which OASIS beats ISOMER.

q	Stale	ISOMER	OASIS-noProj	OASIS	Fresh-init	Fresh	OASIS vs ISOMER	Win frac
1	1.224	1.137	1.271	1.181	1.104	1.000	-3.9%	43.3%
3	1.679	1.298	1.370	1.283	1.146	1.000	+1.2%	53.5%
5	2.004	1.371	1.511	1.348	1.160	1.000	+1.7%	55.0%
10	2.774	1.568	1.647	1.461	1.169	1.000	+6.8%	59.1%
15	3.103	1.735	1.894	1.616	1.167	1.000	+6.8%	56.6%
20	3.124	1.784	1.827	1.590	1.165	1.000	+10.9%	60.9%
25	3.380	1.763	1.911	1.601	1.173	1.000	+9.2%	58.3%
30	3.091	1.790	1.869	1.556	1.174	1.000	+13.0%	60.3%
All	2.415	1.536	1.645	1.446	1.157	1.000	+5.9%	55.9%

5.5. Why the Learned Prior Still Matters After Calibration

The prior-source swap shows that Stage 2 can calibrate many marginal sources. The converse question is whether the learned OASIS prior still adds value once calibration is fixed. We therefore hold the projection operator and feedback constraints fixed and vary only the marginal that initializes projection: stale (ISOMER), learned (OASIS), or post-drift fresh as an upper bound. All variants are scored on held-out future predicates.

Table 5 gives the intended, modest Stage-1 claim. Initializing the same projection from the learned marginal lowers future-predicate Q-error from 1.536 (ISOMER) to 1.446 (OASIS), a 5.9% reduction, with larger gains at stronger drift (+6.8%, +10.9%, and +13.0% at $q = 10$, 20, and 30). The ladder stale-initialized 1.536, learned-initialized 1.446, fresh-initialized 1.157 keeps the roles clear: Stage 1 improves the starting marginal, while Stage 2 is still needed because the raw learned marginal is worse than ISOMER on this held-out projection metric.

At this point the deployed calibration rule and the role of the learned prior are fixed. We next leave the in-distribution generator and ask whether the same rule survives drift mechanisms outside the training distribution.

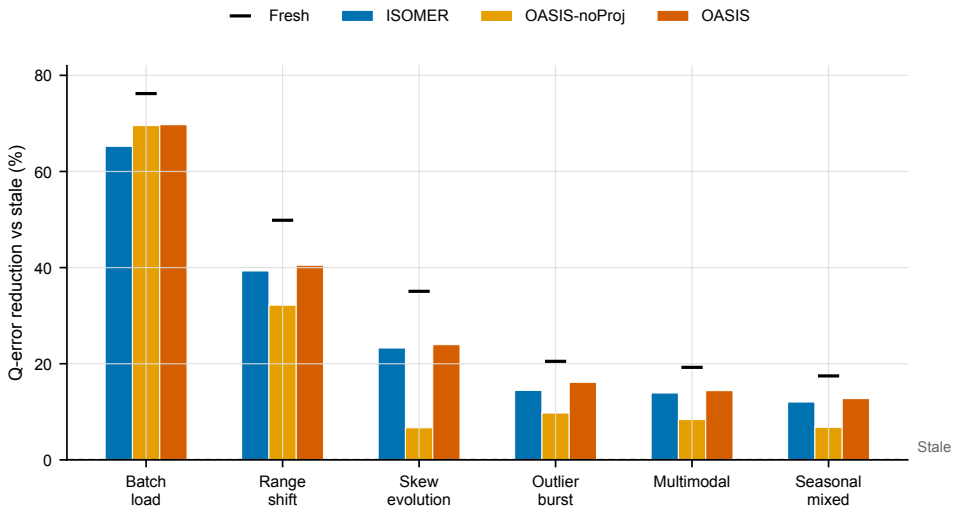

Figure 2: Out-of-distribution drift realism suite. OASIS is trained only on compound drift and evaluated unchanged on deployment-inspired drift families. Bars report the percentage reduction in geometric-mean selectivity Q-error relative to stale statistics for the same drift family; higher is better. The dashed zero line is stale, and black ticks show the fresh post-drift reference.

5.6. Out-of-Distribution Drift Realism

The remaining single-column experiments form a realism ladder. The first step asks whether the learned-prior-plus-calibration path survives drift mechanisms that were not used for training. The main OASIS checkpoint is trained on compound drift, then evaluated unchanged on six deployment-inspired drift families: batch loads, range shifts, skew evolution, outlier bursts, multimodal drift, and seasonal/mixed drift. Each case starts from a stale histogram, interleaves the new drift family with 16 feedback observations, and evaluates future selectivity against the final post-drift distribution. No OOD retraining is used.

Figure 2 shows that the correction path remains useful under unseen drift mechanisms. In the strongest batch-load case, stale Q-error is 4.203, OASIS-noProj reduces it to 1.280, and OASIS further improves to 1.272; across the other families, OASIS remains the best non-fresh method. This is not production-trace validation, but it shows that the gains are not confined to the training drift generator.

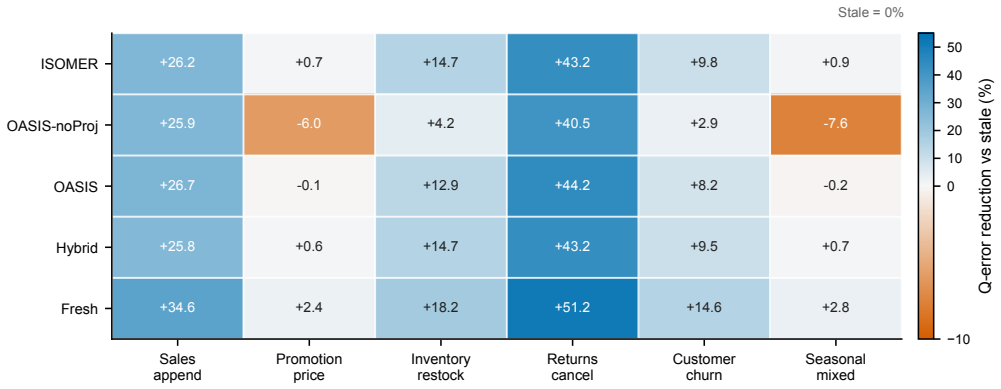

Figure 3: Benchmark-inspired DML trace sanity check. Each trace emits explicit insert/update/delete events anchored to an analytical benchmark maintenance pattern. Cells report the percentage reduction in geometric-mean selectivity Q-error relative to stale statistics for the same trace; higher is better, while negative values indicate regressions below stale. Stale is 0% by definition, and the Fresh row shows the post-drift reference.

5.7. Benchmark-Inspired DML Trace Sanity Check

The second step replaces drift operators with explicit event sequences. The OOD suite changes the drift family, but it still uses synthetic drift operators. We therefore use benchmark-inspired DML streams anchored to analytical table/column maintenance patterns: append-heavy sales dates, promotion price revisions, inventory restocking, returns and cancellations, customer segment churn, and seasonal mixed maintenance. Each trace emits insert, update, and delete counts before a feedback observation is collected. These traces are not production logs or DBMS telemetry; they are a transparent sanity check that the correction layer remains useful when drift is produced by event sequences rather than by the compound training generator.

Figure 3 reinforces the same deployment pattern under event-sequence drift. Across the six traces, stale statistics have aggregate Q-error 1.295; OASIS reduces this to 1.076 and residual-gated Hybrid to 1.072. OASIS-noProj can regress on mild update-heavy traces, again supporting the rule that planners should consume OASIS or residual-gated Hybrid rather than the raw prior.

Table 6: Public production-trace case study using the NASA Kennedy Space Center HTTP request trace. Each request appends one event to an analytics table; stale statistics are built from an early prefix, feedback is collected after later real requests, and future predicate probes include constants drawn from held-out later requests. Values are geometric mean selectivity Q-error. This is a public telemetry trace, not a private DBMS query log.

Trace column	Signal	Growth	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Router	Fresh
Reply bytes	log reply size	800%	1.056	1.059	1.219	1.086	1.090	**1.056**	1.056	1.000
Time of day	request timestamp	800%	1.767	1.254	2.283	1.303	1.275	**1.232**	1.252	1.000
Path length	request path text	800%	1.025	1.057	1.275	1.117	1.097	**1.042**	1.045	1.000
Aggregate	all	–	1.241	1.120	1.525	1.165	1.151	**1.107**	1.114	1.000

5.8. Public Production-Telemetry Trace Case Study

The final step uses a real public trace, while keeping the claim carefully bounded. The benchmark-inspired DML traces above are transparent but synthetic. Because we do not have access to private DBMS production workload logs, we use the public NASA Kennedy Space Center HTTP request trace from the Internet Traffic Archive [23], a standard web-server workload trace also used in workload-characterization research [24]. We treat each request as an append to an analytics/event table and derive three single-column signals from real request fields: log-normalized reply bytes, time of day, and log-normalized path length. Each case builds stale statistics from an early prefix, appends later real requests while collecting predicate feedback, and evaluates future predicate probes that include constants drawn from held-out later requests. This is a public production telemetry trace, not a private DBMS query log or runtime experiment.

Table 6 exposes the deployment boundary the Router is meant to handle. The real append stream is not uniformly friendly to the learned prior: OASIS-noProj regresses in aggregate (1.525 versus stale 1.241) and especially on time-of-day predicates. Stage-2 calibration prevents that failure. Hard projection and consistency-gated soft projection recover most of the loss, and the residual-gated Hybrid/Router variants give the best aggregate results among deployable non-fresh choices (1.107 and 1.114). Router chooses ISOMER or stale in 63.2% of cases and an OASIS variant in 36.8%, which is exactly the intended behavior for a deployment layer: use the learned prior only when it is consistent with the observed feedback window, and fall back to local feedback when the real trace exposes over-extrapolation.

6. Optimizer-Facing Evaluation

The preceding section chooses and stress-tests the marginal statistics object. The next question is propagation: if a downstream estimator or a real planner consumes that calibrated marginal, does the benefit survive, and does the calibration prevent new unsafe decisions? We answer this with increasingly planner-facing consumers: a controlled composition family, a FactorJoin-style join kernel, PostgreSQL statistics injection, a TPC-H runtime sanity check, and sparse/noisy feedback diagnostics. The boundary from Section 1 still holds: the primary evidence is estimate quality and plan *shape*, not execution time. The one exception is the TPC-H study (Section 6.4), a deliberately narrow runtime check whose only claim is that calibrated statistics do not introduce $> 15\%$ median-time regressions; we make no broad runtime-superiority claim.

6.1. Composition-Family Generalization

A natural concern is whether the deployment pattern—OASIS-noProj is an unstable composition input, and feedback-consistency projection is required—is specific to one downstream estimator. We therefore embed OASIS as a drop-in marginal source in a family of six independently designed composition estimators and vary only the marginal input. The family covers the independence/maximum-entropy estimator, iterative proportional fitting (IPF/Sinkhorn) on a two-dimensional grid, and four copulas with distinct dependence structure: Gaussian, Clayton, Gumbel, and Frank. To keep the comparison a clean marginal ablation and to respect the scope boundary that OASIS does not infer dependence, the dependence structure is held fixed and identical for every marginal input: each copula uses one association parameter derived from the data-generating correlation, and IPF seeds its cell-level association from a stale two-dimensional sketch and only reconciles it with each method's corrected one-dimensional marginals. The Archimedean copulas are intentionally mis-specified models for the Gaussian-correlated data, which tests whether the marginal-repair benefit survives an imperfect dependence model.

Table 7 confirms the same pattern across the full family. OASIS-noProj yields weak and inconsistent gains, whereas OASIS improves every estimator by 22.2%–38.1% and Hybrid by 22.7%–38.4%. The calibration benefit therefore does not depend on one dependence model, and it persists under the mis-specified Archimedean copulas.

Table 7: OASIS as a drop-in marginal upgrade across a family of composition estimators. Average joint Q-error (↓) for two-column range predicates; dependence structure is held fixed per estimator and only the marginal input varies. OASIS and Hybrid improve every estimator, whereas OASIS-noProj (the learned stage without the feedback-consistency projection) is weak and unstable, confirming the projection is required across composition methods.

Estimator	Stale	OASIS-noProj	ISOMER	OASIS	Hybrid	Fresh	OASIS-noProj +%	OASIS +%	Hybrid +%
Independence	1.924	1.798	1.191	1.191	**1.185**	1.138	+6.6%	+38.1%	+38.4%
IPF/Sinkhorn-2D	1.668	1.564	1.302	1.280	**1.274**	1.248	+6.3%	+23.3%	+23.6%
Gaussian copula	1.645	1.558	1.251	1.224	**1.216**	1.182	+5.3%	+25.6%	+26.1%
Clayton copula	1.836	1.571	1.310	1.256	**1.251**	1.222	+14.4%	+31.6%	+31.9%
Gumbel copula	1.639	1.535	1.254	1.234	**1.226**	1.192	+6.4%	+24.7%	+25.2%
Frank copula	1.621	1.552	1.291	1.260	**1.253**	1.221	+4.2%	+22.2%	+22.7%

6.2. FactorJoin Integration

The previous composition studies remain single-table. To test whether corrected marginals propagate into a learned *join* cardinality estimator, we embed OASIS in the bin-based join kernel of FactorJoin [25]. For an equi-join $A.k = B.k$, FactorJoin partitions the join-key domain into bins and estimates the join size as $\sum_{\text{bin}} \text{cnt}_A[\text{bin}]\,\text{cnt}_B[\text{bin}]/\text{dv}[\text{bin}]$, where each table's per-bin key frequency $\text{cnt}_T[\text{bin}]$ is read from a single-column histogram of its join-key distribution. We reimplement this kernel and make the per-table key histogram the OASIS-correctable object: when the key distribution drifts, a stale histogram yields wrong per-bin frequencies and a wrong join estimate. The two tables' keys are generated independently, so the only quantity that varies across methods is the quality of each single-column key histogram; the binning, the distinct-values-per-bin assumption, and the row counts are identical for all methods. The metric is join-cardinality Q-error against the exact join size on the drifted data; no query runtime is measured.

Table 8 shows the strongest calibration failure mode. OASIS-noProj is actively harmful, raising aggregate join Q-error above stale (−13.5%). The reason is structural: FactorJoin is bilinear in per-bin mass vectors, so small mass-concentration errors can be amplified. Feedback-consistency projection mitigates this failure: OASIS improves over stale at every tested drift level (12.3%–23.6%), Hybrid improves by 13.4%–23.8%, and both closely track the fresh-marginal input. The more nonlinear the downstream composition, the more essential calibration becomes.

Table 8: OASIS embedded in the FactorJoin bin-based join kernel. Join-cardinality Q-error (↓) for a two-table equi-join whose join-key distributions drift; only the single-column key histogram consumed by FactorJoin varies across methods. OASIS and Hybrid recover most of the stale-to-fresh join error, whereas OASIS-noProj (the learned stage without projection) is actively harmful—it exceeds the stale baseline in this bilinear kernel.

q	Stale	OASIS-noProj	ISOMER	OASIS	Hybrid	Fresh	OASIS +%	Hybrid +%
5	1.323	1.248	1.009	1.010	**1.008**	1.005	+23.6%	+23.8%
10	1.179	1.435	**1.013**	1.031	1.021	1.005	+12.6%	+13.4%
20	1.179	1.447	**1.017**	1.021	1.021	1.005	+13.4%	+13.4%
30	1.178	1.389	1.026	1.034	**1.020**	1.005	+12.3%	+13.4%
All	1.213	1.377	**1.016**	1.024	1.017	1.005	+15.6%	+16.1%

Table 9: Multi-configuration PostgreSQL planner-only statistics-injection evidence. Values are means over configurations; parentheses show one standard deviation across configurations. True rows use `COUNT(*)`; estimated rows and plan shapes use `EXPLAIN (FORMAT JSON)`. No query runtime is measured.

Method	Row QE	Fresh Plan	Recovery	New Deviations
Stale	28.436 (6.585)	56.0% (5.5)	0.0% (0.0)	0.0% (0.0)
ISOMER	2.372 (0.283)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
OASIS-noProj	3.313 (0.228)	88.6% (2.7)	91.4% (3.2)	13.7% (2.0)
OASIS	2.375 (0.275)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
Hybrid	2.374 (0.286)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
Fresh	1.415 (0.095)	100.0% (0.0)	100.0% (0.0)	0.0% (0.0)

6.3. PostgreSQL Planner-Only Statistics Injection

The previous experiments evaluate selectivity and downstream estimator behavior. We next validate that corrected statistics can affect a real DBMS optimizer pipeline. The PostgreSQL experiment creates a synthetic table with fresh current data, builds alternative single-column statistics for `fact.x`, and injects them into `pg_statistic` by copying typed statistics rows from analyzed source tables. For each query, true cardinality is obtained with `COUNT(*)`; PostgreSQL estimated rows and plan shape are obtained with `EXPLAIN (FORMAT JSON)`. The experiment does not measure query runtime.

Table 9 provides the strongest DBMS-facing evidence in this paper. The batch covers two drift directions, two table sizes, and three random seeds, yielding 12 PostgreSQL configurations. Across all query instances, stale statistics produce aggregate row Q-error 27.748 and match the fresh-statistics plan shape on only 56.0% of queries. OASIS-noProj reduces row Q-error to

3.306 and recovers 91.7% of stale/fresh plan-shape disagreements, demonstrating that learned marginal correction propagates into PostgreSQL's native planner. However, the learned stage alone also introduces new plan deviations in 13.8% of cases where stale statistics already matched fresh.

OASIS largely removes this safety issue. It reduces aggregate row Q-error to 2.360, matches the fresh-statistics plan shape on 96.1% of queries, recovers 92.6% of stale/fresh plan-shape disagreements, and reduces new plan deviations to 1.1%. Across configurations, mean row Q-error is 2.375 ± 0.275 for OASIS, 2.372 ± 0.283 for ISOMER, and 3.313 ± 0.228 for OASIS-noProj. Two of these are almost identical—OASIS (2.375) and ISOMER (2.372)—and we explain this directly rather than hide it. On these planner-facing queries the feedback window is dense enough to pin the active constraints, so hard projection onto those constraints converges to nearly the same feedback-consistent marginal whether it is initialized from the learned prior or from the stale histogram: the initialization washes out. The planner is therefore the regime where *calibration*, not the prior, does the work—what matters is that some hard projection is applied (OASIS-noProj, at 3.313, is the only configuration without it and is clearly the worst), not which prior seeds it. The learned prior's advantage instead appears where feedback is too sparse to pin the distribution: future single-column predicates (Section 5.5) and OOD batch-load drift (Section 5.6). This split is exactly the paper's thesis—Stage 2 is the deployment-facing contribution and Stage 1 is a useful but secondary prior—so the near-tie with ISOMER on the planner is expected, not a weakness.

The query-family breakdown gives the same safety picture. Stale statistics disagree with the fresh-statistics plan on 95/336 selection queries, 215/336 join queries, and 134/336 join-with-dimension-filter queries. OASIS matches the fresh plan on 98.5% of selections, 92.3% of joins, and 97.6% of filtered joins. The safety benefit over the learned stage alone is clearest in new deviations: OASIS-noProj creates new plan-shape deviations in 7.5%, 19.8%, and 17.8% of the three families, whereas OASIS reduces these rates to 0.0%, 5.0%, and 0.0%.

Finally, we check that adding Soft to Router introduces no plan-shape regression here. On a six-configuration planner subset (left/right/bimodal shift $\times$ two seeds, 504 queries), the Router is byte-identical to Hybrid—row Q-error 2.234, fresh-plan match 90.1%, new plan deviations 8.17%—because the gate routes to ISOMER on every configuration and never selects soft. For reference, hard projection alone is 2.237/90.9%/6.86% and soft

projection alone is 2.277/91.1%/6.54% on the same subset. The residual gate's preference for ISOMER over hard projection on these planner-facing cases is a property of the existing Hybrid, not of the soft candidate: adding soft to the pool is plan-shape-safe by construction, and the router's single-column and optimizer-facing gains (Section 5.3) come at no planner cost relative to the deployed Hybrid.

6.4. TPC-H DML-Drift Runtime Sanity Check

The planner-only results above show that calibration moves PostgreSQL toward the fresh-statistics plan. The natural follow-up question is what this means for execution time. We answer it on a standard schema with a deliberately narrow check whose claim we state up front and defend below: calibrated statistics *reproduce fresh-statistics behavior*—accuracy, plan shape, and runtime—rather than beat the stale baseline on wall-clock.

We load TPC-H scale factor 10 into PostgreSQL and drift it with refresh-style DML: an RF1-style insert stream appends orders and line items shifted into a new date band, so the stale histograms of `lineitem.l_shipdate` and `orders.o_orderdate` no longer cover the current date range. We then inject single-column statistics for those two columns exactly as in Section 6.3—stale, ISOMER, OASIS-noProj, OASIS, the calibrated router, and post-drift fresh as an oracle—reusing the *same* pretrained OASIS prior with no TPC-H retraining, which also tests cross-schema generalization. For six date-sensitive queries (TPC-H Q1/Q3/Q4/Q6/Q12/Q14 templates) we record the single-column scan row Q-error, the plan shape, and the warm-cache execution time from `EXPLAIN (ANALYZE)` (median of repeated runs, parallelism and JIT disabled). We repeat the whole pipeline over three independent refresh-stream seeds and report mean $\pm$ std.

Table 10 reports the result. The accuracy story is unambiguous and stable across seeds: stale statistics are badly wrong on the drifted date columns (scan Q-error 5.23 ± 0.24), and calibration repairs them on this standard schema with no retraining, with OASIS at 2.02 and the router at 2.01, matching fresh (2.01). The runtime story is the honest one. Calibrated statistics reproduce the fresh-statistics runtime to within a few percent (time-to-fresh ratio 0.96 for OASIS and 1.01 for the router), with small variance across seeds—deploying OASIS makes the optimizer behave, at execution time, as if it had just run a full `ANALYZE`, but without the table scan. The stale wall-clock baseline, by contrast, is not a reliable target: here a stale mis-estimate happens to pick a cheaper-to-run plan, so stale is coincidentally $\approx 1.5\times$ *faster*

Table 10: TPC-H SF10 DML-drift study, aggregated over three independent refresh-stream seeds (mean $\pm$ std; six curated date-sensitive queries each; planner-only statistics injection, PostgreSQL 14). The pretrained OASIS prior is reused with no TPC-H retraining. *Scan Q-err* is the row-estimation error on the drifting date column. *Time/Fresh* and *Time/Stale* are geometric-mean ratios of warm-cache `EXPLAIN ANALYZE` execution time to the fresh- and stale-statistics runs. Calibrated statistics (OASIS, Router) match fresh statistics on both accuracy and runtime (Time/Fresh ≈ 1). The stale wall-clock baseline is not a reliable target: a stale mis-estimate can pick a coincidentally cheaper-to-run plan (Time/Stale > 1 here), whereas in other configurations it picks a much slower one; we therefore make no runtime-superiority claim and report that OASIS reproduces fresh-statistics behavior.

Method	Scan Q-err	Time/Fresh	Time/Stale
Stale	5.23 ± 0.24	0.65 ± 0.07	1.00 ± 0.00
ISOMER	2.02 ± 0.08	0.98 ± 0.07	1.51 ± 0.07
OASIS-noProj	2.66 ± 0.07	1.02 ± 0.02	1.59 ± 0.18
OASIS	2.02 ± 0.10	0.96 ± 0.04	1.49 ± 0.11
Router	2.01 ± 0.10	1.01 ± 0.02	1.57 ± 0.15
Fresh	2.01 ± 0.07	1.00 ± 0.00	1.56 ± 0.17

than fresh, whereas in a larger-drift configuration the same mis-estimate instead produced a plan that ran $1.56\times$ *slower* than fresh. In other words, stale statistics make the optimizer's wall-clock outcome unpredictable in either direction, while calibrated statistics track the fresh-statistics outcome. This is a single-schema check on six queries with three drift seeds, not a runtime benchmark; the claim is reproducibility of fresh-statistics behavior, and we draw no broader runtime-superiority conclusion.

6.5. Feedback Budget and Noise Sensitivity

The TPC-H check is the full-stack sanity check under clean feedback. The last optimizer-facing diagnostic therefore stress-tests the feedback channel itself: how fragile is the deployment rule when the standard 16-observation window is sparser or when observed selectivities are noisy? In the optimizer-decision proxy, consistency projection remains helpful with only two visible observations, and at $K = 16$ Hybrid reaches selectivity Q-error 1.504 with 91.2% join-optimal choices. Under 10% multiplicative feedback noise, Hybrid reaches Q-error 1.565 and keeps the same 90.5% join-optimal rate as OASIS. The detailed curves are in the supplement; the main implication is that residual-based gating and abstention should be part of deployment.

Table 11: Deployment safety checks. Projection and residual-based Hybrid gating are evaluated without oracle/fresh information. P/I/O/S denote OASIS, ISOMER, OASIS-noProj, and stale choices in the residual gate.

Check	Risk signal	Unguarded state	Guarded state	Gate signal
PostgreSQL planner	New plan deviations	OASIS-noProj 13.8%	OASIS 1.1%; FreshPlan 96.1%	projection
PostgreSQL joins	Join new deviations	OASIS-noProj 19.8%	OASIS 5.0%; Recovery 90.7%	projection
Sparse feedback ($K = 2$)	Join-optimal choices	Stale 79.0%	Hybrid 84.6%; NewRisk 2.5%	I 27%, P 57%, O 14%, S 1%
Full feedback ($K = 16$)	Join-optimal choices	Stale 79.0%	Hybrid 91.2%; NewRisk 1.4%	I 37%, P 47%, O 14%, S 2%
10% feedback noise	Join-optimal choices	Stale 79.4%	Hybrid 90.5%; NewRisk 1.7%	I 36%, P 42%, O 18%, S 3%
DML trace sanity	Selectivity Q-error	Stale 1.295	Hybrid 1.072; OASIS 1.076	I 77%, P 20%, O 2%, S 1%

6.6. Deployment Safety Summary

Table 11 collects the safety evidence that determines how OASIS should be deployed. The learned stage alone (OASIS-noProj) improves row-estimation accuracy, but it can introduce plan deviations when the stale planner already agreed with fresh statistics. The projection stage reduces these new deviations from 13.8% to 1.1% in the PostgreSQL batch and from 19.8% to 5.0% on join queries. In the optimizer-decision proxy, residual-gated Hybrid improves join-optimal choices under sparse feedback, full feedback, and 10% feedback noise while keeping new risky decisions near 1–3%. On the benchmark-inspired DML traces, Hybrid improves aggregate selectivity Q-error from 1.295 to 1.072 and selects ISOMER or OASIS in 97% of cases. On the public NASA production-telemetry trace, OASIS-noProj regresses to 1.525 aggregate Q-error, while Hybrid and Router recover to 1.107 and 1.114. These results make the projection stage and residual gating part of the method, rather than optional post-processing.

7. Overhead and Deployment Considerations

Per-correction OASIS inference averages 1.06 ms in our implementation, including tensorization and one forward pass. The PostgreSQL-style statistics-format conversion adds 0.066–0.073 ms, for a combined correction cost of approximately 1.13 ms. Feedback collection adds bounded bookkeeping per filter conjunct and emits one observation tuple per conjunct at query completion. These measurements support the intended use case: maintaining optimizer-facing statistics between scheduled statistics refreshes, not replacing full refreshes after major schema changes or bulk data shifts.

8. Related Work

Cost-based query optimization and selectivity estimation have long relied on compact statistics objects built from data scans [1, 2, 3, 4], with streaming summaries such as KLL sketches providing compact approximate quantiles [26]. Query feedback has also been used for adaptive selectivity estimation [27]. Feedback-driven histograms, including self-tuning histograms, STHoles, SASH, and ISOMER, refine statistics from observed query results [28, 21, 29, 20]. Execution-feedback frameworks can further repair or refine optimizer state over time [30]. OASIS follows this statistics-facing line, but separates prior generation from optimizer-facing calibration and exposes a router that decides when exact feedback consistency is preferable to a softer repair.

Learned cardinality estimators such as NeuroCard, Naru, DeepDB, FACE, MSCN, FLAT, and Iris predict query cardinalities using learned density, representation, or dataset-summary models [31, 32, 33, 34, 35, 36, 37]. Benchmark and deployment studies show both the promise of learned estimators and the difficulty of making them robust across workloads, schemas, and optimizer settings [12, 13, 14, 10]. Recent systems address drift, dynamic workloads, domain adaptation, or production deployment directly [38, 39, 40, 41, 42, 43]. These systems are complementary to OASIS: they typically output point estimates or learned estimator state, while OASIS outputs corrected marginal statistics that can be consumed by an existing CBO.

Copula-based methods and factorized join estimators such as FactorJoin [25] combine one-dimensional marginals with a dependence or join-key model; our composition-family and FactorJoin experiments embed OASIS as the marginal source in exactly these estimators and show that marginal quality and feedback consistency are prerequisites for such downstream use. Related plan-level and robust optimization systems, including mid-query reoptimization, progressive optimization, LEO, Neo, Bao, and Flow-Loss, adjust plan choices, re-plan after observed errors, or train estimates that matter to plans [15, 16, 44, 17, 18, 19, 45, 11]. OASIS operates below that layer by correcting the statistics object itself. A system could combine both approaches: statistics correction supplies better base-table inputs, while plan-level learning handles residual cost-model or join-order effects.

9. Limitations

The evaluation is limited to tested synthetic drift families, benchmark-inspired DML traces, one public production HTTP telemetry trace, and controlled PostgreSQL planner-only scenarios. The benchmark-inspired DML traces make insert/update/delete event streams explicit, but they are not production logs. The NASA trace is real production telemetry, but it is an append-only event-table proxy rather than a private DBMS query workload or statistics history; validating the same correction policy on production database statistics histories and query workload traces remains future work. The PostgreSQL batch covers two drift directions, two table sizes, and three random seeds, so it validates the statistics-to-planner path more robustly than a single case. The TPC-H runtime check (Section 6.4) shows only that calibrated statistics reproduce fresh-statistics execution time across three drift seeds; it is a single schema with six queries and does not establish that correcting statistics is faster than leaving them stale—indeed we observe that a stale mis-estimate can yield a coincidentally faster plan. Runtime in general depends on executor behavior, caching, physical design, system load, and cost-model calibration. OASIS also does not infer multi-column dependence from single-column feedback. When downstream estimates depend on correlations, an external dependence model or multivariate statistics source is still required. Finally, feedback itself can be noisy because of predicate attribution, sampling, or concurrent updates. Our noise diagnostic shows graceful degradation up to 10% multiplicative feedback noise, but deployments should pair correction with residual-based gating, abstention, and rollback policies. Router also gates on feedback residual, which is a proxy for estimate quality rather than for plan-shape risk: on the PostgreSQL subset it prefers ISOMER and so inherits the Hybrid's slightly higher new-deviation rate (8.17% versus 6.86% for OASIS), even though it never routes to Soft there. A plan-shape-aware gating criterion that could route to hard projection over ISOMER on planner-facing cases is left to future work.

10. Conclusion

OASIS repairs stale single-column optimizer statistics from query feedback without rescanning tables or modifying the optimizer. The main lesson is to separate prior generation from optimizer-facing calibration. The learned OASIS MLP supplies a useful prior—under a fixed projection operator, it improves held-out future-predicate Q-error over projection from stale

statistics—but the paper's deployment contribution is the second question: how to make any imperfect prior safe to consume. Raw repaired marginals can be locally inconsistent with the feedback window; downstream estimators amplify that inconsistency, giving weak composition gains and even FactorJoin regressions. Hard feedback projection makes the marginal safe for composition and planner-facing use, improving every member of a six-estimator composition family by 22%–38% and reducing PostgreSQL new plan deviations from 13.8% to 1.1%.

Calibration itself is regime-dependent. OASIS is the right anchor for joins and the DBMS planner, whereas Soft generalizes better on future single-column predicates and the optimizer-decision proxy. The residual-gated Router resolves this split using only deployment-visible feedback residuals: it lowers single-column Q-error from 1.442 to 1.395 and optimizer-proxy selectivity Q-error from 1.446 to 1.396 over OASIS, while reducing to OASIS or ISOMER on the join kernel and PostgreSQL planner where Soft is not selected. The banded projection negative result shows that the improvement is not a consequence of loosening constraints; it comes from choosing the right calibration regime. Finally, the Stage-1 prior-source swap shows that this layer is not tied to the OASIS MLP: the same Stage-2 Router improves stale, heuristic, STHoles, QuickSel-H, and learned priors in aggregate. OASIS is therefore best understood as a reusable feedback-calibrated statistics layer for maintaining optimizer-facing marginals between scheduled `ANALYZE` passes. A three-seed TPC-H runtime check indicates that calibrated statistics reproduce fresh-statistics execution time without the table scan, though we make no broad query-runtime-superiority claim over leaving statistics stale.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, T. G. Price, Access path selection in a relational database management system, in: Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, ACM, 1979, pp. 23–34. doi:10.1145/582095.582099.

[2] S. Chaudhuri, An overview of query optimization in relational systems, in: Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, ACM Press, 1998, pp. 34–43. doi:10.1145/275487.275492.

[3] Y. E. Ioannidis, V. Poosala, Universality of serial histograms, in: Proceedings of the 19th VLDB Conference, 1993, pp. 256–267.

[4] V. Poosala, Y. E. Ioannidis, Selectivity estimation without the attribute value independence assumption, in: Proceedings of the 23rd VLDB Conference, 1997, pp. 486–495.

[5] PostgreSQL Global Development Group, PostgreSQL documentation: Planner statistics, https://www.postgresql.org/docs/current/planner-stats.html, accessed 2026-06-01 (2026).

[6] PostgreSQL Global Development Group, PostgreSQL documentation: Row estimation examples, https://www.postgresql.org/docs/current/row-estimation-examples.html, accessed 2026-06-01 (2026).

[7] PostgreSQL Global Development Group, PostgreSQL documentation: ANALYZE, https://www.postgresql.org/docs/current/sql-analyze.html, accessed 2026-06-01 (2026).

[8] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proceedings of the VLDB Endowment 9 (3) (2015) 204–215.

[9] G. Moerkotte, T. Neumann, G. Steidl, Preventing bad plans by bounding the impact of cardinality estimation errors, Proceedings of the VLDB Endowment 2 (1) (2009) 982–993.

[10] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cai, Y. Zhang, G. Li, Cardinality estimation in DBMS: A comprehensive benchmark evaluation, Proceedings of the VLDB Endowment 15 (4) (2021) 752–765.

[11] K. Lee, A. Dutt, V. R. Narasayya, S. Chaudhuri, Analyzing the impact of cardinality estimation on execution plans in microsoft SQL server, Proceedings of the VLDB Endowment 16 (11) (2023) 2871–2883. doi:10.14778/3611479.3611494.

[12] X. Wang, C. Qu, W. Wu, J. Wang, Q. Zhou, Are we ready for learned cardinality estimation?, Proceedings of the VLDB Endowment 14 (9) (2021) 1640–1654. doi:10.14778/3461535.3461552.

[13] J. Sun, J. Zhang, Z. Sun, G. Li, N. Tang, Learned cardinality estimation: A design space exploration and a comparative evaluation, Proceedings of the VLDB Endowment 15 (1) (2021) 85–97. doi:10.14778/3485450.3485459.

[14] K. Kim, J. Jung, I. Seo, W.-S. Han, K. Choi, J. Chong, Learned cardinality estimation: An in-depth study, in: Proceedings of SIGMOD, ACM, 2022, pp. 1214–1227. doi:10.1145/3514221.3526154.

[15] N. Kabra, D. J. DeWitt, Efficient mid-query re-optimization of sub-optimal query execution plans, in: Proceedings of SIGMOD, 1998, pp. 106–117.

[16] V. Markl, V. Raman, D. E. Simmen, G. M. Lohman, H. Pirahesh, Robust query processing through progressive optimization, in: Proceedings of SIGMOD, ACM, 2004, pp. 659–670. doi:10.1145/1007568.1007642.

[17] M. Stillger, G. Lohman, V. Markl, M. Kandil, LEO – DB2's learning optimizer, in: Proceedings of the 27th VLDB Conference, 2001, pp. 19–28.

[18] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, Proceedings of the VLDB Endowment 12 (11) (2019) 1705–1718.

[19] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of SIGMOD, 2022, pp. 1275–1288.

[20] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. Lang, V. Raman, Consistent selectivity estimation via maximum entropy, The VLDB Journal 16 (2) (2007) 169–188.

[21] N. Bruno, S. Chaudhuri, L. Gravano, STHoles: A multidimensional workload-aware histogram, in: Proceedings of SIGMOD, 2001, pp. 211–222.

[22] Y. Park, S. Zhong, B. Mozafari, QuickSel: Quick selectivity learning with mixture models, in: Proceedings of SIGMOD, 2020, pp. 1017–1033.

[23] Internet Traffic Archive, NASA-HTTP: Two months of HTTP logs from the Kennedy Space Center WWW server, https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html, accessed 2026-06-01.

[24] M. F. Arlitt, C. L. Williamson, Web server workload characterization: The search for invariants, in: Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 1996, pp. 126–137. doi:10.1145/233013.233034.

[25] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, S. Madden, FactorJoin: A new cardinality estimation framework for join queries, Proceedings of the ACM on Management of Data (SIGMOD) 1 (1) (2023) 1–27.

[26] N. Ivkin, J. Nelson, H. Yu, V. Braverman, KLL: Approximately optimal streaming quantile estimation, Proceedings of the ACM on Management of Data 2 (1) (2019) 1–23.

[27] C. M. Chen, N. Roussopoulos, Adaptive selectivity estimation using query feedback, in: Proceedings of the 24th ACM SIGMOD International Conference on Management of Data, ACM Press, 1994, pp. 161–172. doi:10.1145/191839.191874.

[28] A. Aboulnaga, S. Chaudhuri, Self-tuning histograms: Building histograms without looking at data, in: Proceedings of SIGMOD, 1999, pp. 181–192.

[29] L. Lim, M. Wang, J. S. Vitter, SASH: A self-adaptive histogram set for dynamically changing workloads, in: Proceedings of the 29th VLDB Conference, 2003, pp. 369–380. doi:10.1016/B978-012722442-8/50040-9.

[30] S. Chaudhuri, V. R. Narasayya, R. Ramamurthy, A pay-as-you-go framework for query execution feedback, Proceedings of the VLDB Endowment 1 (1) (2008) 1141–1152. doi:10.14778/1453856.1453977.

[31] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, I. Stoica, NeuroCard: One cardinality estimator for all tables, Proceedings of the VLDB Endowment 14 (1) (2020) 61–73.

[32] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, I. Stoica, Deep unsupervised cardinality

estimation, Proceedings of the VLDB Endowment 13 (3) (2019) 279–292.

[33] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, C. Binnig, DeepDB: Learn from data, not from queries!, Proceedings of the VLDB Endowment 13 (7) (2020) 992–1005.

[34] J. Wang, C. Chai, J. Liu, G. Li, FACE: A normalizing flow based cardinality estimator, Proceedings of the VLDB Endowment 15 (1) (2021) 72–84. doi:10.14778/3485450.3485458.

[35] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, A. Kemper, Learned cardinalities: Estimating correlated joins with deep learning, in: CIDR, 2019.

[36] R. Zhu, Z. Wu, Y. Han, K. Zeng, A. Pfadler, Z. Qian, J. Zhou, B. Cui, FLAT: Fast, lightweight and accurate method for cardinality estimation, Proceedings of the VLDB Endowment 14 (9) (2021) 1489–1502. doi:10.14778/3461535.3461539.

[37] Y. Lu, S. Kandula, A. C. König, S. Chaudhuri, Pre-training summarization models of structured datasets for cardinality estimation, Proceedings of the VLDB Endowment 15 (3) (2022) 414–426. doi:10.14778/3494124.3494127.

[38] B. Li, Y. Lu, S. Kandula, Warper: Efficiently adapting learned cardinality estimators to data and workload drifts, in: Proceedings of SIGMOD, ACM, 2022, pp. 2146–2160. doi:10.1145/3514221.3517914.

[39] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, M. Alizadeh, Robust query driven cardinality estimation under changing workloads, Proceedings of the VLDB Endowment 16 (6) (2023) 1520–1533. doi:10.14778/3583140.3583164.

[40] P. Li, W. Wei, R. Zhu, B. Ding, J. Zhou, H. Lu, ALECE: An attention-based learned cardinality estimator for SPJ queries on dynamic workloads, Proceedings of the VLDB Endowment 17 (2) (2023) 197–210. doi:10.14778/3626292.3626302.

[41] Z. Wang, Q. Zeng, N. Wang, H. Lu, Y. Zhang, CEDA: Learned cardinality estimation with domain adaptation, Proceedings of the VLDB Endowment 16 (12) (2023) 3934–3937. doi:10.14778/3611540.3611589.

[42] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[43] Y. Han, H. Wang, L. Chen, Y. Dong, X. Chen, B. Yu, C. Yang, W. Qian, ByteCard: Enhancing ByteDance's data warehouse with learned cardinality estimation, in: Companion of the 2024 International Conference on Management of Data, ACM, 2024, pp. 41–54. doi:10.1145/3626246.3653376.

[44] B. Babcock, S. Chaudhuri, Towards a robust query optimizer: A principled and practical approach, in: Proceedings of SIGMOD, ACM, 2005, pp. 119–130. doi:10.1145/1066157.1066172.

[45] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, M. Alizadeh, Flow-loss: Learning cardinality estimates that matter, Proceedings of the VLDB Endowment 14 (11) (2021) 2019–2032. doi:10.14778/3476249.3476259.